



AuditWare

X

Swapboard

Smart Contract Security Review

Performed by **Auditware** for **ECF**



auditware.io



audit_wizard



Review Summary

The Auditware team has performed a security review on Swapboard.

Swapboard is a minimalist, trustless OTC bulletin board for peer-to-peer ERC20 token swaps on Ethereum. The protocol implements an immutable orderbook with zero fees, no admin functions, and atomic execution guarantees.

Audit commit hash: [aecea2d](#)

The following items were in scope for the audit:

- Smart contract vulnerabilities in `contracts/src/`
- Frontend vulnerabilities that could lead to fund loss
- Subgraph data integrity issues

The following items were explicitly defined out of scope:

- Known behaviors documented in "Trust Assumptions" and "Known Limitations" sections
- Third-party dependencies (OpenZeppelin, ethers.js) - report to upstream maintainers
- Theoretical attacks without proof of concept or economic viability analysis
- Social engineering, phishing, or UI spoofing attacks
- Issues requiring compromised private keys
- Network-level attacks (eclipse attacks, BGP hijacking)
- Vulnerabilities in the testnet deployment or test tokens (TestToken.sol)
- Frontend issues that do not lead to fund loss (cosmetic bugs, UX issues)
- Subgraph indexing delays or temporary data inconsistencies
- Gas optimization suggestions (unless they enable griefing attacks)

No critical issues were found during the engagement. All findings can be shown in detail in the Findings Overview below.

As with most protocols, it is highly recommended to:

- Conduct regular and thorough internal and external audits to identify and mitigate vulnerabilities.
- Achieve the highest possible test coverage, ideally approaching 100%.
- Continuing to maintain comprehensive and up-to-date documentation, ensuring code-to-docs compliance.
- Participate in auditing contests and/or a bug bounty program.
- Implement on-chain monitoring for real-time threat detection and mitigation.



Findings Overview

Finding	Component	Risk Level	Retest Status
<u>AW-M-01: The fill confirmation names and sizes both tokens from self-reported metadata only</u>	Frontend	Medium	Unmitigated
<u>AW-L-01: Exported CSVs carry unescaped cell values that spreadsheets execute as formulas</u>	Frontend	Low	Unmitigated
<u>AW-L-02: Mock mode ships to production and a link switches it on for every later visit</u>	Frontend	Low	Unmitigated



AW-M-01: The fill confirmation names and sizes both tokens from self-reported metadata only

Severity: **Medium**

Status: **Unmitigated**

Locations:

- [frontend/app.js:2197-2224](#) - `buildTokenCell` renders `token.symbol` as the token's name
- [frontend/app.js:3051-3053](#) - the confirmation headline is built from the same self-reported symbol and the indexer-stored `decimals`

Description:

The last screen before the wallet prompt reads “You will send N SYMBOL and receive M SYMBOL”. Both symbols are whatever the token contracts return, and both amounts are formatted with the `decimals` the indexer stored when it first saw the token. The order table and the order detail show the same. The create form treats this as a risk, marks anything outside its short verified list as `[unknown token]` and warns that fake tokens can steal funds; the fill path carries no address, no link and no mark.

A maker deploys a token whose `symbol()` returns WBTC or USDC and posts an order selling it for real USDC at an attractive price. A taker sees `[Fill]`, reads the confirmation, approves and fills - real USDC leaves, the worthless token arrives. The only cue is the copy button and the address in a hover title, which the confirmation does not show.

The amounts are no safer than the names as the transaction sends `availableB` in base units, while the sentence divides that same value by the indexer's `decimals`. Reproduced locally with the indexer answering `decimals`: 18 for a six-decimal token: the confirmation read `You will send 0 TKB and receive 40 TKA`, the taker confirmed, and the transaction paid 90 TKB. The indexer stores `decimals` once, falls back to 18 when the call reverts, and never re-reads it, so a token that reverts on `decimals()` at indexing time or changes it afterwards controls what the confirmation says.



Any maker can do this with no privilege, and the confirmation is exactly where the taker decides.

Attack scenario:

1. A demo token self-reports the symbol WETH and lists 5 of it for 2,500 USDC. The order table shows the self-reported symbol verbatim:

```
const wrap = document.createElement("span");
wrap.style.whiteSpace = "nowrap";

const cgUrl = coinGeckoUrl(token.address);
if (cgUrl) {
  const link = document.createElement("a");
  link.href = cgUrl;
  link.target = "_blank";
  link.rel = "noopener noreferrer";
  link.textContent = token.symbol;
  wrap.appendChild(link);
} else {
  const span = document.createElement("span");
  span.textContent = token.symbol;
  wrap.appendChild(span);
}
```

[frontend/app.js:2209-2224](#)

	Trade ID ▾	Maker	Offered Token	Offered Size
[Fill]	9001	0x000010010000100100001001000010010000100100001001 	=HYPERLINK("http://evil.example/claim","Claim airdrop") 	3
[Fill]	9000	0x0000100000001000000010000000100000001000 	WETH 	5

Trade 9000's symbol cell claims WETH, 5 offered for 2,500 USDC

2. The fill confirmation is built from the same self-reported symbol - the last line before the wallet prompt reads:

```
summary.textContent =
  `You will send ${formatAmount(remainingB, order.tokenB.decimals)}
  ${order.tokenB.symbol} ` +
  `and receive ${formatAmount(remainingA, order.tokenA.decimals)}
  ${order.tokenA.symbol} in return.`;
```

[frontend/app.js:3051-3053](#)



VC Fundraising: 0
 Token ICO: 0
The ticker is \$ETH, motherfucker

Popular Pairs: [\[DAI/USDC\]](#) [\[WETH/UNI\]](#) [\[AAVE/stETH\]](#) [\[USDT/DAI\]](#) [\[SHIB/LINK\]](#) [\[USDT/LINK\]](#) [\[UNI/USDC\]](#) [\[SHIB/WBTC\]](#) [\[stETH/USDC\]](#) [\[stETH/MATIC\]](#)

Filters:
 Offered: All Wanted: All
 Token: Token:

All Trades: ☐ Open Trades: ☒ Filled Trades: ☐ Cancelled Trades: ☐ Watched: ☐ My Orders: ☐

	Trade ID	Maker	Offered Token	Offered Si
[Fill]	9001	0x000010010000100	im", "Claim airdrop")	3
[Fill]	9000	0x000010000000100		5
[Fill]	97	0x000010020000100		9
[Fill]	96	0x000010010000100		8.5
[Fill]	95	0x0000100000001000000010000000100000001000	ETH	8
[Fill]	94	0x0000100e0000100e0000100e0000100e0000100e	ETH	7.5
[Fill]	93	0x0000100d0000100d0000100d0000100d0000100d	ETH	7
[Fill]	92	0x0000100c0000100c0000100c0000100c0000100c	ETH	6.5
[Fill]	91	0x0000100b0000100b0000100b0000100b0000100b	ETH	6
[Fill]	90	0x0000100a0000100a0000100a0000100a0000100a	ETH	5.5
[Fill]	89	0x0000100900001009000010090000100900001009	ETH	5

Fill Order #9000

You will send 2,500 USDC and receive 5 WETH in return.

Confirm Cancel

Estimating gas...

MOCK MODE ACTIVE - Data is simulated

The fill confirmation names the scam token WETH

Risk:

A taker who trusts the confirmation buys a worthless token believing it is WETH - the self-reported symbol is the only identity shown at the moment of consent.

A scam token priced near the market rate is filled as if it were the real asset. The actor is anyone who can deploy a token and place an order; the loss is the full USDC sent.

Recommendation:

- Carry the create form's rule onto the fill path: mark tokens outside the verified list with **[unknown token]** in the table, the detail and the confirmation, and show the short address next to the symbol on the confirmation.
- Render the same warning line the create modal shows when either side is unverified.
- Read **decimals()** from the token at confirmation time and refuse to show an amount when it disagrees with the indexer.
- Call the already-declared **getOrder/canFill** before building the fill confirmation and require the indexed token addresses and remainings to match on chain - the frontend



declares both in its ABI but never invokes them, so the indexer is the sole source for what is displayed and signed.

- Consider requiring an extra click to fill an order whose tokens are not on the list.



AW-L-01: Exported CSVs carry unescaped cell values that spreadsheets execute as formulas

Severity: **Low**

Status: **Unmitigated**

Locations:

- [frontend/app.js:2218](#) - the order table renders the symbol verbatim
- [frontend/app.js:4472-4482](#) - **Export My Orders** joins cells into the CSV unescaped

Description:

Export CSV writes token symbols and other subgraph text into cells after escaping only commas, quotes and newlines. **Export My Orders** escapes nothing. A cell that begins with =, +, - or @ is evaluated by Excel and LibreOffice as a formula.

A maker's token with `symbol()` set to `=HYPERLINK("https://evil","USDC")` puts a live link into every export that lists the order. Older spreadsheet versions run DDE payloads such as `=cmd|'/C calc'!A0` from the same cell.

Attack scenario:

1. A token's `symbol()` is rendered verbatim in the order table. A demo token's symbol is a spreadsheet formula, shown as-is:

```
if (cgUrl) {
  const link = document.createElement("a");
  link.href = cgUrl;
  link.target = "_blank";
  link.rel = "noopener noreferrer";

  wrap.appendChild(link);
  link.textContent = token.symbol;
} else {
  const span = document.createElement("span");
  wrap.appendChild(span);
  span.textContent = token.symbol;
}
```

[frontend/app.js:2213-2224](#)



	Trade ID ▾	Maker	Offered Token	Offered Size
[Fill]	9001	0x0000100100001001000010010000100100001001	=HYPERLINK("http://evil.example/claim","Claim airdrop")	3

Trade 9001's symbol cell is a formula, shown as-is

2. **Export My Orders** joins the raw cells into the CSV without escaping - the formula lands in the Offered Token cell:

```
o.orderId,  
status,  
o.tokenA.symbol,  
amtA,  
o.tokenB.symbol,  
amtB,  
o.createdAt,  
o.taker || "",  
];  
});  
const csvContent = [headers.join(","), ...rows.map((r) => r.join(",")).join("\n");  
frontend/app.js:4472-4482
```

swapboard-my-orders-0xf39Fd6.csv - the Offered Token cell of trade 9001, unescaped:

```
Trade ID,Status,Offered Token,Offered Amount,Wanted Token,Wanted Amount,Created,Taker  
9001,Open,=HYPERLINK("http://evil.example/claim","Claim airdrop"),3,USDC,900,1791300382,  
79,Open,LINK,30,DAI,3000,1791295542,
```

Export My Orders writes the raw =HYPERLINK formula into the Offered Token cell



Risk:

A maker can plant a formula in every export of the book. Anyone who exports and opens the file in Excel, Numbers or LibreOffice executes it: HYPERLINK exfiltration and DDE payloads are the classic consequences.

The actor is any token deployer; the cost is one order.

Recommendation:

- Prefix any cell starting with =, +, -, @, tab or carriage return with a single quote.
- Apply the same escaper to `exportMyOrders`, which currently escapes nothing.

AW-L-02: Mock mode ships to production and a link switches it on for every later visit

Severity: **Low**

Status: **Unmitigated**

Locations:

- [frontend/index.html:468](#) - `mock.js` loads unconditionally in the production bundle
- [frontend/mock.js:64-76](#) - the URL parameter returns before the hostname check
- [frontend/mock.js:70-76](#) - the choice is persisted to `localStorage`

Description:

`mock.js` is part of the production bundle and `?mock=true` on the real origin writes `swapboard_mock=true` to `localStorage`, which wins on every following visit until the reader knows to pass `?mock=false`. In mock mode the board shows fabricated orders, prices and a **Mock Wallet** with a fake balance, and creates and fills succeed without any transaction.

Anyone who can get a user to open a link puts that user's browser into this state on the production origin. Real wallets are suppressed in mock mode, so no transaction can be signed by mistake; what remains is a persistent, shareable, misleading view of the board.

Attack scenario:

1. `mock.js` loads on every page, production included. The production deployment at `swap.ethcf.org` serves the file (HTTP 200) with the code path below.

```
<script src="lib.js"></script>
<script src="mock.js"></script>
<script src="app.js"></script>
frontend/index.html:467-469
```



2. The URL parameter returns before the hostname check, so a single link forces mock mode on regardless of origin - the same production hostname, before and after the link:

```
function shouldEnableMock() {
  // 1. Check URL parameter (highest priority)
  const urlParams = new URLSearchParams(window.location.search);
  const mockParam = urlParams.get("mock");
  if (mockParam !== null) {
    const enabled = mockParam === "true" || mockParam === "1";
  }
}
frontend/mock.js:64-69
```

[Fill]	143	0x9a50a078d80f36e38edfae85affa2b8ab458e2c9	ZAMM	100	WETH	1	\$ --	0.01 WETH / ZAMM
[Fill]	142	0x9a50a078d80f36e38edfae85affa2b8ab458e2c9	ZAMM	100	USDC	100	\$ --	\$1.00 / ZAMM
[Fill]	140	0x023688771e6b8a4099667dd0159eb59da9fe43b3	USDC	100	eUSDSr	100	\$99.99	\$1.00 / eUSDSr
[Fill]	64	0x012a8def817ff5d69165fd3982e141ee8c571df0	WETH	0.1	PERPIR	1.000.000	\$270.62	10.00M

Clean visit on a production hostname, no banner

[Fill]	96	0x0000100100001001000010010000100100001001	WETH	8.5	USDC	20,400	\$29,750	\$2,4 WE
[Fill]	95	0x000010000000100000001000000010000001000	WETH	8	USDC	19,000	\$28,000	\$2,3 WE

MOCK MODE ACTIVE - Data is simulated
Disable

?mock=true on the same hostname, mock mode forced on

3. The param choice is written to **localStorage**, so the state persists across later visits:

```
// Persist the choice
try {
  localStorage.setItem(STORAGE_KEY, String(enabled));
} catch (e) {
  // localStorage may be unavailable
}
return enabled;
frontend/mock.js:70-76
```

[Fill]	96	0x0000100100001001000010010000100100001001	WETH	8.5	USDC	20,400	\$29,750	\$2,4 WE
[Fill]	95	0x000010000000100000001000000010000001000	WETH	8	USDC	19,000	\$28,000	\$2,3 WE

MOCK MODE ACTIVE - Data is simulated
Disable

Clean visit after, the banner is back, the choice persisted

Risk:

One link to a visitor switches their production session to simulated data: orders, balances and confirmations are fake while the user believes they are live.

A phished visitor can be shown a favorable book and a fake fill, and it persists until they notice the banner.

Recommendation:

- Keep the mock out of the production build, or gate it behind a build flag rather than a URL parameter. If a preview must stay, do not persist the choice.





This report was produced by Auditware for ETHCF based solely on the information provided by them. Auditware provides no guarantees as to the accuracy of the contents of this report and does not make any guarantees that following the advice within will prevent security incidents or issues.

Auditware is available for questions or comments about any of the contents of this report. We can be reached at <https://auditware.io/> or by email at joe@auditware.io.

